

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №3
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
«Розробка інтерфейсу користувача»

Виконав:
студентка групи ІМ-52
Матюшенко Анастасія Юріївна
Номер в списку - 11

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета роботи: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.

Завдання:

1. Створити у середовищі **JetBrains Rider** проєкт типу **WPF Application** (.NET 9) з ім'ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати проєкт і отримати виконуваний файл програми.
4. Проаналізувати та прокоментувати результати й вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

- **Зберігання фігур:** Динамічний масив вказівників на 112 елементів ($12 + 100$), оскільки $12 \bmod 3 = 0$.
- **Гумовий слід:** Суцільна лінія чорного кольору ($12 \bmod 4 = 0$).
- **Прямокутник:**
 - Ввід: По двом протилежним кутам ($12 \bmod 2 = 0$).
 - Відображення: Чорний контур із жовтим заповненням ($12 \bmod 5 = 2$; $12 \bmod 6 = 0$).
- **Еліпс:**
 - Ввід: Від центру до одного з кутів охоплюючого прямокутника ($12 \bmod 2 = 0$).
 - Відображення: Чорний контур без заповнення ($12 \bmod 5 = 2$).
- **Позначка поточного типу об'єкта:** Відображається в меню (метод `OnInitMenuPopup`) ($12 \bmod 2 = 0$).
- **Toolbar:** Додано панель кнопок з іконками та підказками (tooltips), що дублюють підпункти меню "Об'єкти".

Вихідні тексти програми:

Shape.cs

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab3
{
    public enum ShapeType
    {
        Point,
        Line,
        Rect,
        Ellipse
    }

    public abstract class Shape
```

```

    {
        protected double x1, y1, x2, y2;

        public void Set(double startX, double startY, double endX, double
endY)
        {
            x1 = startX;
            y1 = startY;
            x2 = endX;
            y2 = endY;
        }

        public abstract UIElement CreateElement();

        public abstract void Update(UIElement element);

        public void Draw(Canvas canvas)
        {
            canvas.Children.Add(CreateElement());
        }

        public virtual void ApplyRubberStyle(System.Windows.Shapes.Shape
element)
        {
            element.Stroke = Brushes.Black;
            element.StrokeThickness = 1;
            element.StrokeDashArray = null;
            element.Fill = null;
        }
    }

    public class PointShape : Shape
    {
        private const double Size = 4;

        public override UIElement CreateElement()
        {
            var ellipse = new Ellipse
            {
                Width = Size,
                Height = Size,
                Fill = Brushes.Black
            };
            Update(ellipse);
            return ellipse;
        }

        public override void Update(UIElement element)
        {
            Canvas.SetLeft(element, x1 - Size / 2);
            Canvas.SetTop(element, y1 - Size / 2);
        }

        public override void ApplyRubberStyle(System.Windows.Shapes.Shape
element)
        {
            element.Stroke = Brushes.Black;
            element.StrokeThickness = 1;
            element.Fill = Brushes.Black;
        }
    }

    public class LineShape : Shape
    {

```

```

        public override UIElement CreateElement()
        {
            var line = new Line
            {
                Stroke = Brushes.Black,
                StrokeThickness = 2
            };
            Update(line);
            return line;
        }

        public override void Update(UIElement element)
        {
            if (element is Line line)
            {
                line.X1 = x1;
                line.Y1 = y1;
                line.X2 = x2;
                line.Y2 = y2;
            }
        }
    }

    public class RectShape : Shape
    {
        public override UIElement CreateElement()
        {
            var rect = new Rectangle
            {
                Stroke = Brushes.Black,
                Fill = Brushes.Yellow,
                StrokeThickness = 2
            };
            Update(rect);
            return rect;
        }

        public override void Update(UIElement element)
        {
            if (element is Rectangle rect)
            {
                rect.Width = Math.Abs(x2 - x1);
                rect.Height = Math.Abs(y2 - y1);
                Canvas.SetLeft(rect, Math.Min(x1, x2));
                Canvas.SetTop(rect, Math.Min(y1, y2));
            }
        }
    }

    public class EllipseShape : Shape
    {
        public override UIElement CreateElement()
        {
            var ellipse = new Ellipse
            {
                Stroke = Brushes.Black,
                Fill = null,
                StrokeThickness = 2
            };
            Update(ellipse);
            return ellipse;
        }

        public override void Update(UIElement element)

```

```

        {
            if (element is Ellipse ellipse)
            {
                double dx = Math.Abs(x2 - x1);
                double dy = Math.Abs(y2 - y1);
                ellipse.Width = dx * 2;
                ellipse.Height = dy * 2;
                Canvas.SetLeft(ellipse, x1 - dx);
                Canvas.SetTop(ellipse, y1 - dy);
            }
        }
    }
}

```

Editor.cs

```

using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;

namespace Lab3
{
    public abstract class Editor
    {
        public abstract void OnMouseDown(Canvas canvas, MouseButtonEventArgs e);
        public abstract void OnMouseMove(Canvas canvas, MouseEventArgs e);
        public abstract void OnMouseUp(Canvas canvas, MouseButtonEventArgs e);
    }

    public class ShapeEditor : Editor
    {
        private const int Zh = 12;
        private readonly int N = Zh + 100;

        private readonly List<Shape> pcshape;
        private int shapeCount = 0;

        private double x1, y1, x2, y2;
        private bool isDrawing = false;

        private Shape tempShape;
        private UIElement previewElement;

        public ShapeType CurrentShapeType { get; set; } = ShapeType.Line;

        public ShapeEditor()
        {
            pcshape = new List<Shape>(N);
        }

        public override void OnMouseDown(Canvas canvas, MouseButtonEventArgs e)
        {
            if (shapeCount >= N) return;

            var pos = e.GetPosition(canvas);
            x1 = x2 = pos.X;
            y1 = y2 = pos.Y;
            isDrawing = true;
            canvas.CaptureMouse();
        }
    }
}

```

```

        tempShape = CreateShape(CurrentShapeType);
        tempShape.Set(x1, y1, x2, y2);

        previewElement = tempShape.CreateElement();
        if (previewElement is System.Windows.Shapes.Shape wpfShape)
        {
            tempShape.ApplyRubberStyle(wpfShape);
        }
        canvas.Children.Add(previewElement);
    }

    public override void OnMouseMove(Canvas canvas, MouseEventArgs e)
    {
        if (!isDrawing) return;

        var pos = e.GetPosition(canvas);
        x2 = pos.X;
        y2 = pos.Y;

        tempShape.Set(x1, y1, x2, y2);
        tempShape.Update(previewElement);
    }

    public override void OnMouseUp(Canvas canvas, MouseButtonEventArgs e)
    {
        if (!isDrawing) return;

        var pos = e.GetPosition(canvas);
        x2 = pos.X;
        y2 = pos.Y;
        isDrawing = false;
        canvas.ReleaseMouseCapture();

        canvas.Children.Remove(previewElement);
        previewElement = null;

        tempShape.Set(x1, y1, x2, y2);
        tempShape.Draw(canvas);

        pcsshape.Add(tempShape);
        tempShape = null;
    }

    private static Shape CreateShape(ShapeType type) => type switch
    {
        ShapeType.Point => new PointShape(),
        ShapeType.Line => new LineShape(),
        ShapeType.Rect => new RectShape(),
        ShapeType.Ellipse => new EllipseShape(),
        _ => new LineShape()
    };
}

```

MainWindow.xaml

```

<Window x:Class="Lab3.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Лабораторна робота №3 - Матюшенко Анастасія" Height="450"
        Width="800">
    <DockPanel>
        <Menu DockPanel.Dock="Top">

```

```

        <MenuItem Header="Файл">
            <MenuItem Header="Вихід" Click="Exit_Click"/>
        </MenuItem>
        <MenuItem Header="Об'єкти" x:Name="objectsMenu"
SubmenuOpened="ObjectsMenu_SubmenuOpened">
            <MenuItem x:Name="pointMenuItem" Header="Крапка"
Click="Point_Click"/>
            <MenuItem x:Name="lineMenuItem" Header="Лінія"
Click="Line_Click"/>
            <MenuItem x:Name="rectMenuItem" Header="Прямокутник"
Click="Rectangle_Click"/>
            <MenuItem x:Name="ellipseMenuItem" Header="Еліпс"
Click="Ellipse_Click"/>
        </MenuItem>
        <MenuItem Header="Довідка">
            <MenuItem Header="Про програму" Click="About_Click"/>
        </MenuItem>
    </Menu>
    <ToolBar DockPanel.Dock="Top">
        <ToggleButton x:Name="pointButton" Width="32" Height="32"
ToolTip="Крапка" Click="Point_Click">
            <Canvas Width="20" Height="20">
                <Ellipse Width="6" Height="6" Fill="Black"
Canvas.Left="7" Canvas.Top="7"/>
            </Canvas>
        </ToggleButton>
        <ToggleButton x:Name="lineButton" Width="32" Height="32"
ToolTip="Лінія" Click="Line_Click">
            <Canvas Width="20" Height="20">
                <Line X1="3" Y1="17" X2="17" Y2="3" Stroke="Black"
StrokeThickness="2"/>
            </Canvas>
        </ToggleButton>
        <ToggleButton x:Name="rectButton" Width="32" Height="32"
ToolTip="Прямокутник" Click="Rectangle_Click">
            <Canvas Width="20" Height="20">
                <Rectangle Width="16" Height="12" Canvas.Left="2"
Canvas.Top="4"
                    Stroke="Black" StrokeThickness="2"
Fill="Yellow"/>
            </Canvas>
        </ToggleButton>
        <ToggleButton x:Name="ellipseButton" Width="32" Height="32"
ToolTip="Еліпс" Click="Ellipse_Click">
            <Canvas Width="20" Height="20">
                <Ellipse Width="16" Height="12" Canvas.Left="2"
Canvas.Top="4"
                    Stroke="Black" StrokeThickness="2"/>
            </Canvas>
        </ToggleButton>
    </ToolBar>

    <Canvas x:Name="drawingCanvas" Background="White" ClipToBounds="True"
        MouseLeftButtonDown="drawingCanvas_MouseLeftButtonDown"
        MouseMove="drawingCanvas_MouseMove"
        MouseLeftButtonUp="drawingCanvas_MouseLeftButtonUp"/>
</DockPanel>
</Window>

```

MainWindow.xaml.cs

```

using System.Windows;
using System.Windows.Input;

```

```

namespace Lab3
{
    public partial class MainWindow : Window
    {
        private readonly ShapeEditor shapeEditor = new ShapeEditor();

        public MainWindow()
        {
            InitializeComponent();
            SetShapeType(ShapeType.Line);
        }

        private void SetShapeType(ShapeType type)
        {
            shapeEditor.CurrentShapeType = type;

            pointButton.IsChecked = type == ShapeType.Point;
            lineButton.IsChecked = type == ShapeType.Line;
            rectButton.IsChecked = type == ShapeType.Rect;
            ellipseButton.IsChecked = type == ShapeType.Ellipse;
        }

        private void Exit_Click(object sender, RoutedEventArgs e) =>
        Application.Current.Shutdown();

        private void Point_Click(object sender, RoutedEventArgs e) =>
        SetShapeType(ShapeType.Point);
        private void Line_Click(object sender, RoutedEventArgs e) =>
        SetShapeType(ShapeType.Line);
        private void Rectangle_Click(object sender, RoutedEventArgs e) =>
        SetShapeType(ShapeType.Rect);
        private void Ellipse_Click(object sender, RoutedEventArgs e) =>
        SetShapeType(ShapeType.Ellipse);

        private void ObjectsMenu_SubmenuOpened(object sender, RoutedEventArgs
e)
        {
            var type = shapeEditor.CurrentShapeType;
            pointMenuItem.IsChecked = type == ShapeType.Point;
            lineMenuItem.IsChecked = type == ShapeType.Line;
            rectMenuItem.IsChecked = type == ShapeType.Rect;
            ellipseMenuItem.IsChecked = type == ShapeType.Ellipse;
        }

        private void About_Click(object sender, RoutedEventArgs e)
        {
            MessageBox.Show("Лабораторна робота №3\nВиконала: Матюшенко
Анастасія\nЖ = 12",
                "Довідка", MessageBoxButton.OK, MessageBoxImage.Information);
        }

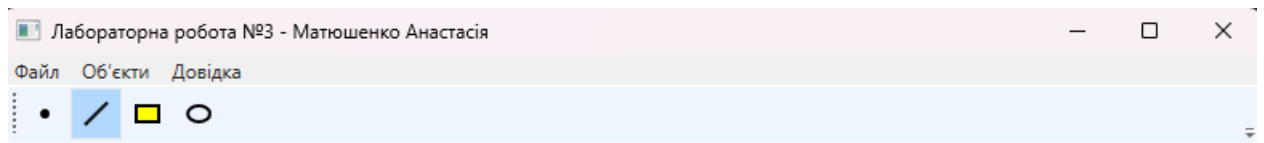
        private void drawingCanvas_MouseLeftButtonDown(object sender,
MouseButtonEventArgs e)
            => shapeEditor.OnMouseDown(drawingCanvas, e);

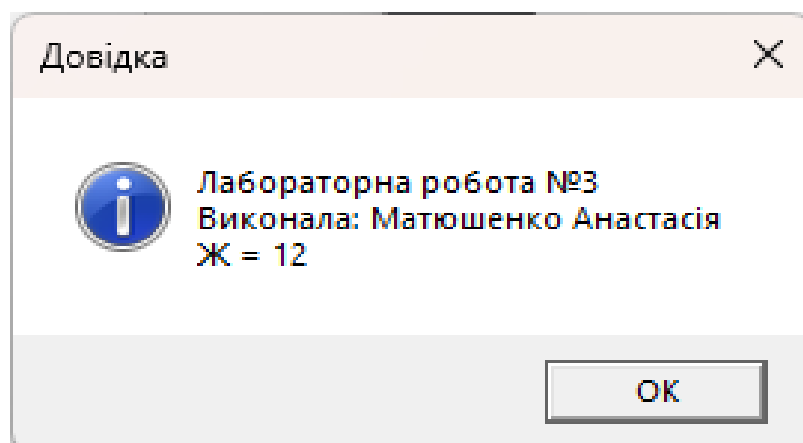
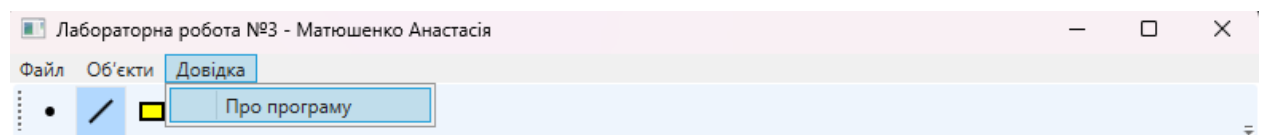
        private void drawingCanvas_MouseMove(object sender, MouseEventArgs e)
            => shapeEditor.OnMouseMove(drawingCanvas, e);

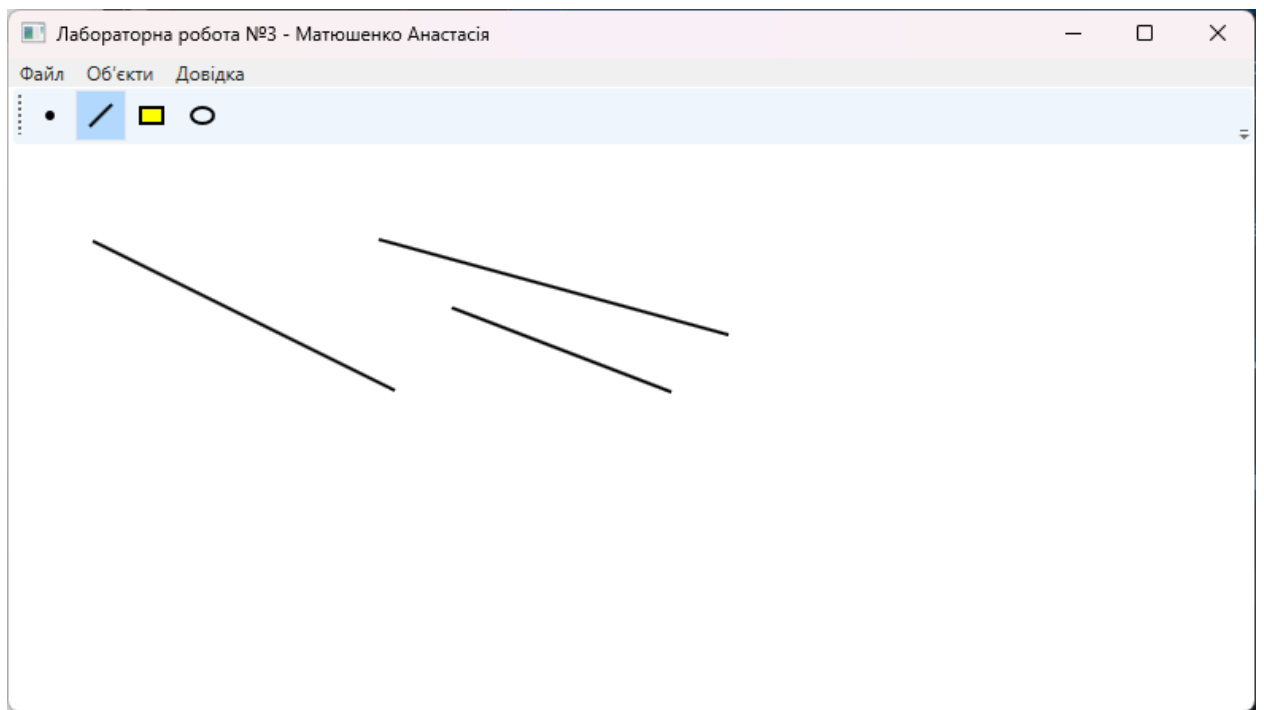
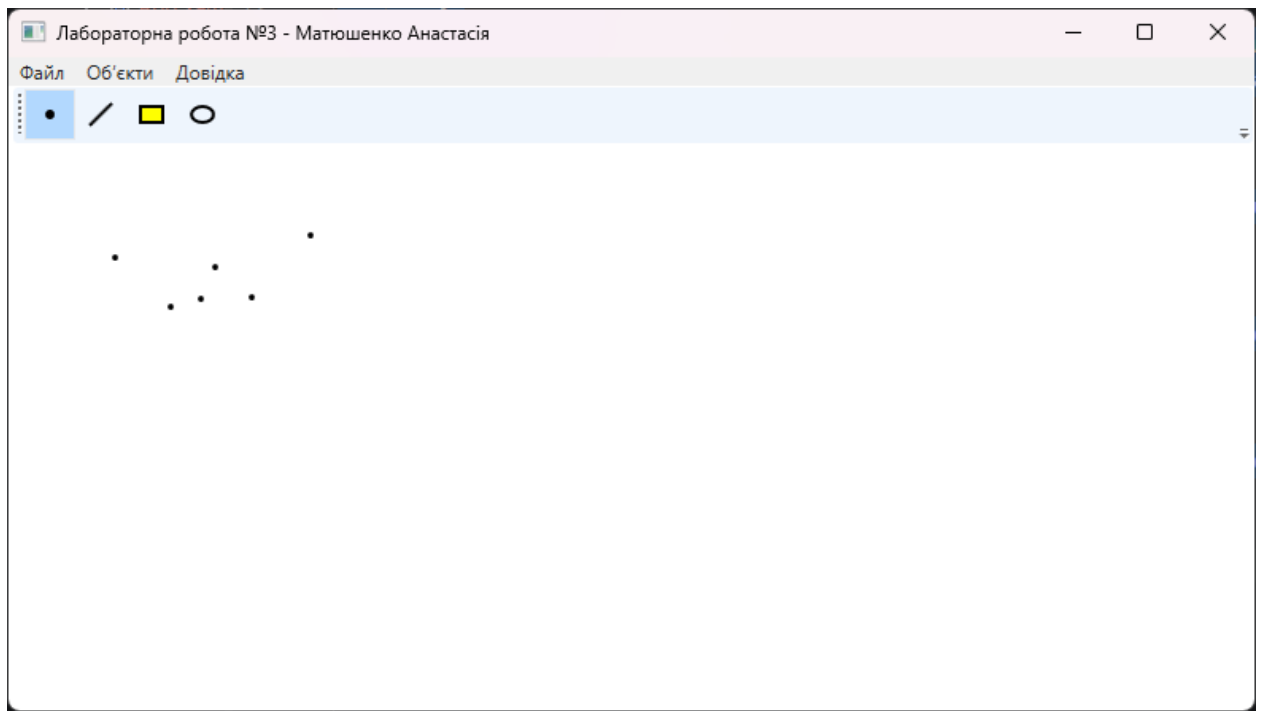
        private void drawingCanvas_MouseLeftButtonUp(object sender,
MouseButtonEventArgs e)
            => shapeEditor.OnMouseUp(drawingCanvas, e);
    }
}

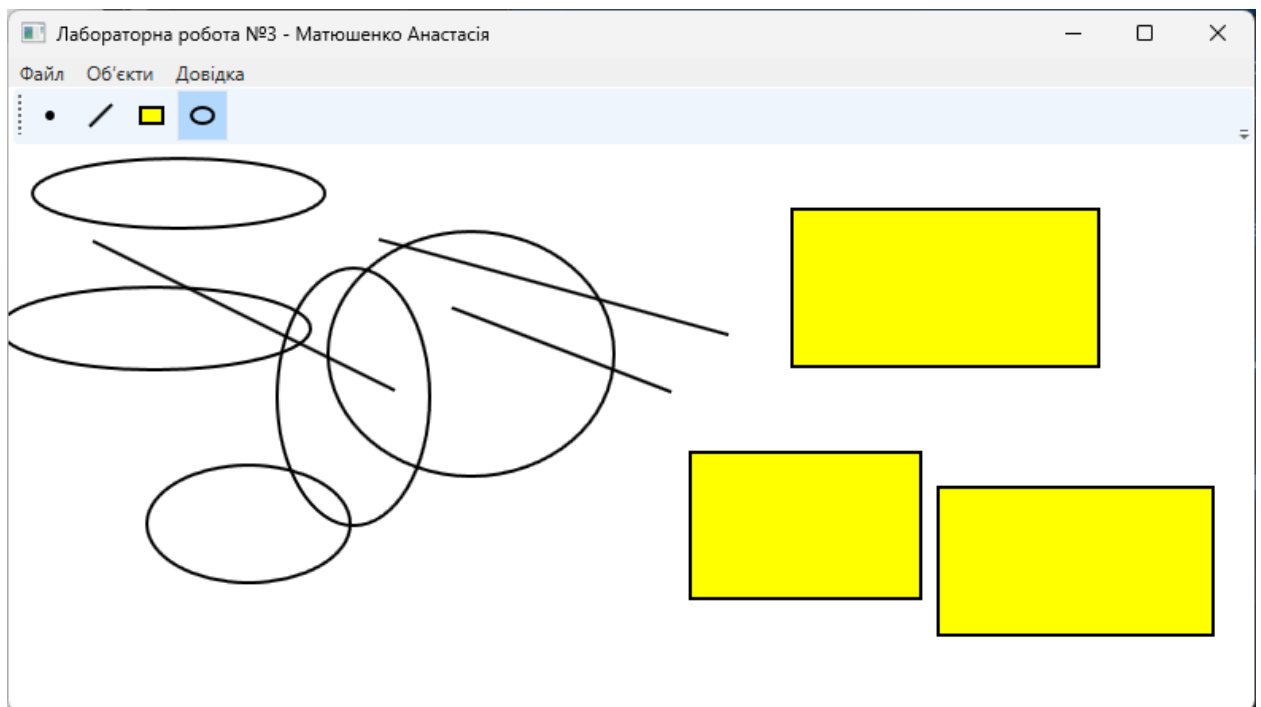
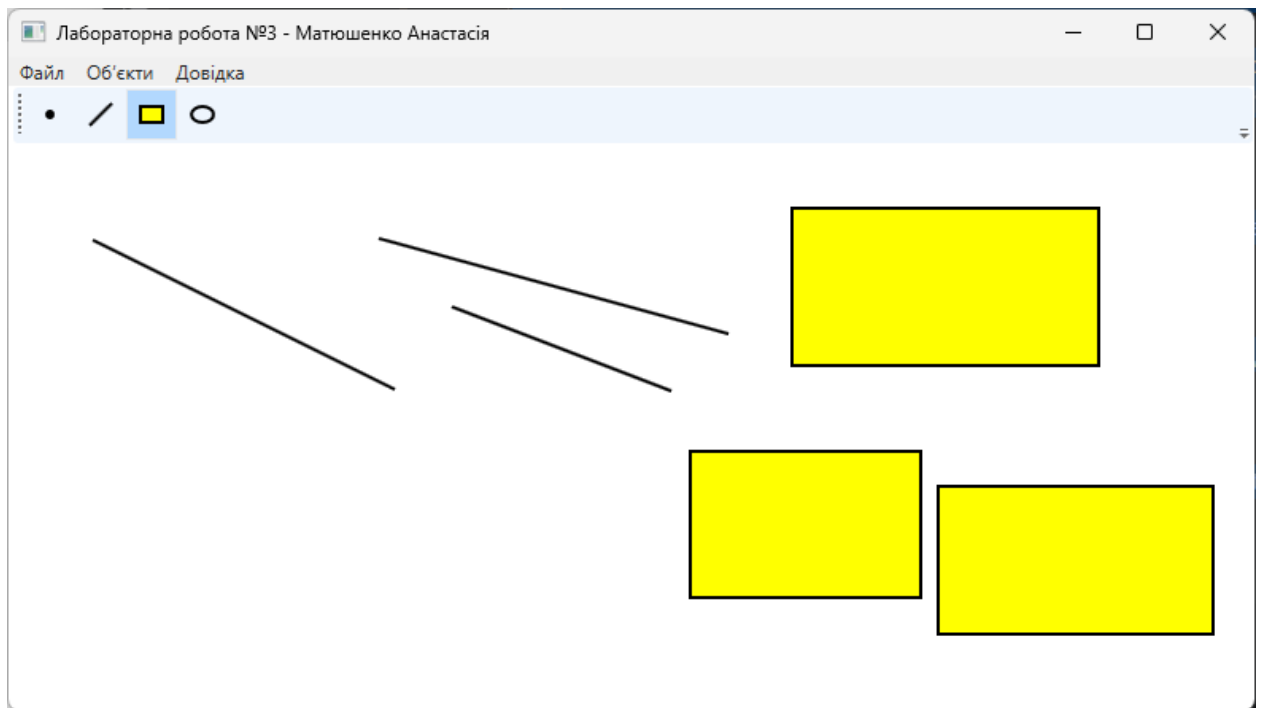
```

Скріншоти:









Висновки: у ході лабораторної роботи було доопрацьовано графічний редактор об'єктів шляхом додавання панелі інструментів (Toolbar) з кнопками-іконками, що дублюють пункти меню "Об'єкти", а також забезпечено узгоджену роботу меню та Toolbar через єдиний метод зміни поточного типу фігури.